

Ex No: 2c

IMPLEMENTATION OF BACKWARD CHAINING

Aim:

To implement backward chaining.

Procedure:

Initialize a dictionary of base facts and a list of rules represented as (conclusion, conditions) tuples.

Define the backward_chaining recursive function to evaluate if a specified goal is satisfiable.

Check if the target goal exists directly in the facts dictionary and return its boolean value.

Iterate through the rules to match the goal; recursively call backward_chaining on each condition to verify if all required conditions hold true.

Define the target goal, invoke backward_chaining(goal, facts, rules), and print whether the goal can be achieved.

Program:

Facts: The knowledge base containing information about facts

```
facts = {
```

```
    'a': True,
```

```
    'b': True,
```

```
    'c': False
```

```
}
```

Rules: List of rules where each rule is a tuple (conclusion, conditions)

```
rules = [
```

```
    ('d', ['a', 'b']), # d can be concluded if a and b are true
```

```
    ('e', ['b', 'c']), # e can be concluded if b and c are true
```

```
    ('f', ['d', 'e']) # f can be concluded if d and e are true
```

```
]
```

Backward Chaining Function

```
def backward_chaining(goal, facts, rules):
```

```
    # Check if the goal is already a fact
```

```
    if goal in facts:
```

```
        return facts[goal]
```

```

        # Check if the goal can be concluded from any rule
    for rule in rules:
        head, body = rule
        if head == goal:
            # Check if all the conditions in the body of the rule are true
            if all(backward_chaining(fact, facts, rules) for fact in body):
                return True

    # If the goal cannot be proved
    return False

# Example Usage:
goal = 'f'

if backward_chaining(goal, facts, rules):
    print(f"The goal '{goal}' can be achieved.")
else:
    print(f"The goal '{goal}' cannot be achieved.")

```

Output:

Expected Output for the Goal 'f' :

The goal 'f' cannot be achieved.

Expected Output for the Goal 'd' :

The goal 'd' can be achieved.

```
Facts: The knowledge base containing information about facts
('a', 'True')
('b', 'True')
('c', 'False')

Rules: List of rules where each rule is a tuple (conclusion, conditions)
('a', [('b', 'c')]) # a can be concluded if b and c are true
('c', [('b', 'c')]) # c can be concluded if b and c are true
('f', [('d', 'e')]) # f can be concluded if d and e are true

Backward Chaining Function
Goal: f
The goal 'f' cannot be achieved.

Example Usage:
goal = 'f'
The goal 'f' cannot be achieved.

Check if the goal can be concluded from any rule
Rule: ('a', [('b', 'c')])
Result: The goal 'a' is achievable.
```